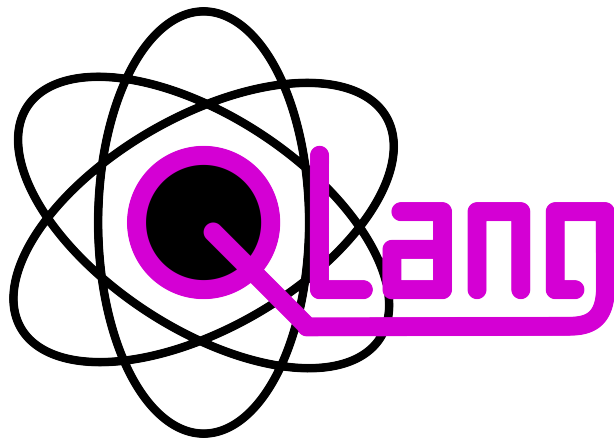




QLang Programming Language

Language Specification & Documentation

Created by:

Kamil Widzyk
Maciej Świątek
Jan Wida

Contents

1	Introduction	3
1.1	Overview	3
1.2	Quick Example	3
1.3	Design Principles	3
1.3.1	Classical and Quantum Code in One Language	3
1.3.2	Structured Error Reporting	4
1.3.3	Modular Structure via Places	4
2	Language Basics	5
2.1	Program Structure	5
2.2	Data Types	5
2.3	Variables	5
2.3.1	Declaration Syntax	5
2.3.2	Arrays	6
2.3.3	Constants	6
2.3.4	The # Size Getter	6
2.4	Operators	7
2.4.1	Arithmetic Operators	7
2.4.2	Unary Operators	7
2.4.3	Comparison and Logical Operators	7
2.4.4	Increment and Decrement	7
2.4.5	Compound Assignment Operators	7
2.4.6	Assignment Expression	7
2.4.7	The reset Operator	8
2.4.8	String Formatting Operator	8
2.4.9	The Parent Operator ^	8
2.5	Control Flow	8
2.5.1	Conditional Statements	8
2.5.2	For Loop	9
2.5.3	While Loop	9
2.5.4	Iterate Loop	10
2.5.5	Break and Continue	10
2.6	Functions	10
2.6.1	Named Arguments	11
2.6.2	Variadic Parameters	11
2.7	Input and Output	12
2.7.1	Output	12
2.7.2	Input	12

3	Quantum Computing	14
3.1	Quantum Types: state and obs	14
3.2	Primitive Gates	14
3.3	Measurement	14
3.4	Example: Quantum Teleportation	15
3.5	Example: Superdense Coding	16
4	Advanced Control Flow and Scope Management	18
4.1	The Parent Operator ^ and Scope Hierarchy	18
4.1.1	Scope in QLang	18
4.1.2	How ^ Works	18
4.2	Advanced Array Structures and Indexing	19
4.2.1	Simple Indexing and Negative Indices	19
4.2.2	Range Indexing	19
4.2.3	List Indexing	19
4.3	...	19

1 Introduction

QLang is a high-level programming language designed for quantum computing. It provides a simple and intuitive syntax for expressing quantum algorithms and operations, making it accessible to both beginners and experienced programmers in the field of quantum computing.

1.1 Overview

A QLang program is made up of top-level statements and **place** declarations. A place is QLang's primary unit of encapsulation, similar to a class or module in classical languages. Each place can define functions and execute statements. The special *global place*, code written outside any place, acts as a default place called **global**.

The language supports four built-in variable types: **state**, **obs**, **num**, and **text**. Quantum-specific operations, gates such as **H**, **X**, **CNOT**, and the **measure** keyword, are first-class constructs in QLang, not library calls. QLang is interpreted. Source files (extension **.ql**) are parsed by an ANTLR4-generated parser and executed by the QLang interpreter written in Python.

1.2 Quick Example

The following complete QLang program creates a Bell state (a maximally entangled two-qubit state), measures both qubits, and prints the results:

```
place Alice {  
    state a, b;  
  
    H a;           // Put qubit 'a' into superposition  
    CNOT a -> b;   // Entangle 'a' with 'b' - this creates a Bell state  
  
    obs ma = measure a;  
    obs mb = measure b;  
  
    println("Measurement A: %d" % ma);  
    println("Measurement B: %d" % mb);  
    // Both will always agree: both 0 or both 1  
}
```

After running, both measurements will always agree, both **0** or both **1**, which is the defining property of a Bell state. This example demonstrates how QLang lets you express a complete quantum protocol in just a few lines.

1.3 Design Principles

1.3.1 Classical and Quantum Code in One Language

QLang does not separate quantum and classical concerns into different APIs or files. A variable of type **obs** (classical integer) and a variable of type **state** (qubit) can appear side-by-side in the same scope. Measurement, when invoked on a **state**, collapses the qubit to a classical value (**obs**) and returns a **bool** indicating the outcome: **false** if the qubit collapsed to $|0\rangle$, **true** if it collapsed to $|1\rangle$.

1.3.2 Structured Error Reporting

When the interpreter encounters an error, whether a syntax mistake, an undefined variable, or an illegal runtime operation, it displays a detailed error box that highlights the exact source location, shows surrounding context lines, and explains the cause.

1.3.3 Modular Structure via Places

QLang organizes code into units called **places**. Each **place** has its own name, its own variables, and runs independently from other places. Places can communicate with each other using the built-in **send** and **receive** instructions, which transfer both classical and quantum data. This makes it natural to write distributed programs, such as quantum communication protocols, directly in QLang.

2 Language Basics

This chapter covers the fundamental building blocks of QLang: how programs are structured, the available data types, variables, operators, and control flow constructs.

2.1 Program Structure

A QLang source file (extension `.ql`) consists of a sequence of top-level items. Each item is one of the following:

- A **place** declaration.
- A function declaration (at global place).
- A statement (at global place).

Items are executed in the order they appear. The global place is the implicit default place and is always present. When a named place is declared, its body executes when the program starts.

2.2 Data Types

QLang has four built-in variable types, designed to cover both classical and quantum computation:

Type	Description
state	A quantum state (qubit or qubit register). Represents a quantum system that can be in superposition and can be entangled with other qubits. Collapsed to a classical value via the measure keyword.
obs	A classical observation. A scalar obs stores a boolean value; a sized register obs x[N] stores an N-bit integer. Used for branching, control, and classical post-processing after measurement.
num	A universal numeric type. It dynamically handles both integers and floating-point numbers, automatically adapting its internal representation based on the assigned value and performed operations. Used for all classical numerical computations.
text	A string of characters. Used for textual data, print formatting, and string operations.

2.3 Variables

Variables are declared by specifying a type, a name, an optional bit-width (for **obs** and **state**), and an optional initial value.

2.3.1 Declaration Syntax

```
num pi = 3.14159;           // floating-point number
num count = 10;             // num accepts integer values too
obs result[16];             // 16-bit integer register, default value 0
obs counter[8] = 0;         // 8-bit integer register, initialized to 0
```

```
text greeting = "Hello!"; // string
```

2.3.2 Arrays

Any variable type can be declared as an array by appending one or more size specifiers in square brackets. Arrays are zero-indexed. A size can be a fixed integer literal or an expression evaluated at runtime:

```
num matrix[4][4]; // two-dimensional 4x4 array

// Size inferred automatically from the initialiser
num a[?] = [1, 2, 3, 4, 5]; // size is 5, inferred from the list
println(#a); // 5

// Runtime-computed size
num n = 8;
num dynamic[n]; // size determined at runtime
```

2.3.3 Constants

The **const** keyword can be applied to any variable declaration to prevent reassignment after initialisation. Variables can be declared as mutable first and then locked with the **const** keyword. Once a variable has been marked as constant, it cannot be reassigned anymore. Attempting to modify a constant at runtime produces an error.

This allows patterns such as:

```
const obs MAX_SIZE[8] = 100;
const num EPSILON = 0.0001;
const PI = 3.14159;

num x = 10;
x++;
const x;
```

For example, **num x = 10; x++; const x;** starts with a mutable variable, updates it, and then freezes it so later assignments are rejected.

2.3.4 The # Size Getter

The **#** prefix operator returns the declared length of an array or the length of a **text** string:

```
num items[?] = [10, 20, 30, 40, 50];
println(#items); // 5
text message = "hello";
println(#message); // 5
```

2.4 Operators

2.4.1 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo
**	Exponentiation

2.4.2 Unary Operators

Operator	Description
+x	Unary plus, returns the value unchanged
-x	Unary minus, negates the value
!x	Logical NOT; returns 1 if x is zero, 0 otherwise

2.4.3 Comparison and Logical Operators

Operator	Description
==	Equal to
!=	Not equal to
<, >, <=, >=	Relational comparisons
&&	Logical AND
	Logical OR

2.4.4 Increment and Decrement

```
num i = 0;
i++;    // post-increment
++i;    // pre-increment
i--;    // post-decrement
--i;    // pre-decrement
```

2.4.5 Compound Assignment Operators

In addition to plain assignment (**=**), QLang supports compound operators: **+=**, **-=**, ***=**, **/=**, **%=**, ****=**. The operators **&=** and **|=** are logical compound assignments equivalent to **x = x && y** and **x = x || y** respectively.

All compound assignment operators return the new value of the variable, so they can be used inside expressions.

2.4.6 Assignment Expression

Plain assignment (**=**) is also an expression and returns the assigned value. Assigning to a non-variable expression is legal but has no effect and simply returns the right-hand side value.

2.4.7 The **reset** Operator

The **reset** operator restores a variable to its initial value as declared, or to the type default if no initial value was given. It works on scalars, strings, and multi-dimensional arrays. It returns the value after reset.

```
// reset scalar
num x = 5;
x = 99;
reset x;
println(x); // 5

// reset text
text s = "hello";
s = "world";
reset s;
println(s); // hello

// reset array
num arr[2] = [1, 2];
arr[0] = 99;
reset arr;
println(arr[0]); // 1
```

2.4.8 String Formatting Operator

The **%** operator, when the left operand is a **text** string, performs printf-style interpolation:

```
num x = 42;
num y = 3.14159;
println("The answer is %d" % x);
println("x = %d, y = %.2f" % [x, y]); // list of values

text fmt = "Hello, %s!";
println(fmt % "world");
```

2.4.9 The Parent Operator **^**

The **^** operator explicitly accesses a variable in an outer scope. Multiple carets chain upward, one level per caret. See Section 4.1 for a full description.

2.5 Control Flow

2.5.1 Conditional Statements

QLang supports standard **if/else if/else** branching:

```

num x = 75;

if (x > 100) {
    println("Large");
} else if (x > 50) {
    println("Medium");
} else {
    println("Small");
}

// if without braces
if (x > 0) println("Positive");

// Ternary operator
text label = x > 50 ? "big" : "small";
println(label);

```

2.5.2 For Loop

The **for** loop iterates over a numeric range with an optional step. The syntax is **for <var> from <start> to <end> step <step>**. The loop variable is created automatically and is only accessible inside the loop body. When **step** is omitted, the interpreter infers it automatically: **+1** if **end > start**, **-1** otherwise.

```

// ascending for loop
for i from 1 to 5 {
    println(i); // 1 2 3 4 5
}

// descending for loop
for i from 5 to 1 {
    println(i); // 5 4 3 2 1
}

// custom step for loop
for i from 0 to 10 step 2 {
    println(i); // 0 2 4 6 8
}

```

2.5.3 While Loop

The **while** loop repeats its body as long as the condition is non-zero or **true**.

```

// while loop

```

```

num x = 0;
while (x < 5) {
    println(x);
    x++;
}
// 0 1 2 3 4

```

2.5.4 Iterate Loop

The **iterate** loop traverses every element of an array. The current element is bound to a named variable. An optional **index** clause provides a second variable that holds the current index.

```

// iterate over array
num arr[4] = [10, 20, 30, 40];
iterate arr as val {
    println(val); // 10 20 30 40
}

// iterate with index
iterate arr as val index i {
    println("%d: %d" % [i, val]);
    // 0: 10
    // 1: 20
    // 2: 30
    // 3: 40
}iterate_loop

```

2.5.5 Break and Continue

break exits the nearest enclosing loop immediately. **continue** skips the rest of the current iteration and proceeds to the next one.

2.6 Functions

Functions are declared with the **function** keyword. Parameters are typed, can have default values, and may be arrays. Supported parameter types are **num**, **obs**, **state**, **text**, and **any**. A parameter of type **any** accepts a value of any type and infers the actual type at the call site. A function returns a value with the **return** statement.

```

function add(num a, num b) {
    return a + b;
}

function greet(text name = "World") {
    println("Hello, " + name + "!");
}

```

```

function power(num base, num exp) {
    return base ** exp;
}

greet(); // Hello, World!
greet("Alice"); // Hello, Alice!
println(add(3, 4)); // 7
println(power(exp = 3, base = 2)); // 8

```

2.6.1 Named Arguments

Arguments can be passed by name, which allows them to be supplied in any order and makes call sites more self-documenting:

```

function rect(num width, num height = 1) {
    return width * height;
}

// Positional arguments
println(rect(5, 3)); // 15

// Mixed: positional + named
println(rect(5, height = 3)); // 15

// All named, any order
println(rect(height = 4, width = 6)); // 24

// Using default value
println(rect(7)); // 7

```

2.6.2 Variadic Parameters

A function can accept a variable number of arguments using the `...` prefix, for example **function sum(...values)**. Variadic arguments do not have a type written before the name and may mix values of different types. Inside the function they are available as an array:

```

function sum(...values) {
    num s = 0;
    for i from 0 to #values {
        s += values[i];
    }
    return s;
}

```

```
println(sum(1, 2, 3));           // 6
println(sum(10, 20, 30, 40));   // 100

// Regular parameters can precede the variadic one
function prefixed(num x, ...rest) {
    println("x=" + x + " count=" + #rest);
}

prefixed(5, 10, 15);           // x=5 count=2
```

2.7 Input and Output

2.7.1 Output

QLang provides three output statements:

- **print(...)** — prints values without a trailing newline.
- **println(...)** — prints values followed by a newline.
- **debug(x)** — prints detailed diagnostic information, including the internal type, shape, and for quantum states the gate history and stabilizer representation.

```
// Basic output
print("no newline");
println(" with newline");
println();           // blank line

// Multiple arguments
println(1, 2, 3);    // 1 2 3

// Printf-style formatting
num val = 42;
println("The answer is %d" % val);
println("x = %d, y = %.2f" % [val, 3.14]);
```

2.7.2 Input

The **input** statement reads a value from the user into a variable. An optional constraint limits the accepted input. For numeric variables the constraint specifies a value range; for **text** variables it specifies the allowed length in characters. The constraint can be written with the range notation **min..max** or the **range()** function. The interpreter keeps prompting until a valid value is entered.

```
obs age[8];
input(age);           // read any value
```

```
obs score[8];  
input(score, 0..100);           // accept only values in range [0, 100]  
input(score, range(0, 100));    // equivalent syntax
```

3 Quantum Computing

This chapter introduces the quantum-specific features of QLang: the built-in quantum types, the primitive quantum gates available in the language, how measurement works, and two canonical example protocols: quantum teleportation and superdense coding.

3.1 Quantum Types: **state** and **obs**

QLang exposes two first-class types for quantum programming:

- **state**: a quantum register (one or more qubits). A **state** can be put into superposition and entangled with other **states** using the built-in gate primitives. A variable of type **state** represents a live quantum system that only yields a classical value when explicitly measured. Quantum states cannot be read, assigned, or printed directly.
- **obs**: a classical observation. A scalar **obs** stores a boolean value; a sized register **obs x[N]** stores an N-bit integer. Used for branching, control, and classical post-processing after measurement.

A variable declaration can specify arrays and bit-widths. **state q[2]** declares a two-qubit register; **obs result[8]** declares an 8-bit classical register.

3.2 Primitive Gates

QLang provides primitive quantum gates as first-class language constructs. Each gate has a short uppercase name and a lowercase alias. The supported gates are listed below.

Gate	Alias	Description
H	superpose	Hadamard gate, creates superposition on a single qubit.
S	shift	Phase gate, introduces a phase shift on a single qubit.
X	not	Pauli-X, flips the computational basis state.
Y	dual_not	Pauli-Y, combines a bit flip and a phase flip.
Z	phase_not	Pauli-Z, applies a relative phase flip between $ 0\rangle$ and $ 1\rangle$.
CNOT	entangle	Controlled-NOT, entangles two qubits: syntax CNOT a -> b .
CZ	entangle_phase	Controlled-Z, applies a phase flip conditioned on the control qubit.

Single-qubit gates act on one **state** variable in-place. Multi-qubit gates use the arrow syntax (**->**) to indicate the control-to-target relationship.

3.3 Measurement

Measurement forces a **state** to collapse into a classical outcome. The **measure** keyword returns a boolean value (**false** for $|0\rangle$, **true** for $|1\rangle$), which is automatically mapped to integer **0** or **1** when assigned to an **obs** variable. Measuring multiple qubits at once produces an **obs** array.

QLang also provides **measureX**, which applies a Hadamard gate before measuring, effectively measuring in the X basis rather than the Z basis.

The following example puts a qubit into superposition, measures it, and prints the result:

```

state q;
H q;
obs result[1];
result = measure q;
println(result);

```

Measurement is nondeterministic when applied to a superposed or entangled state.

3.4 Example: Quantum Teleportation

Quantum teleportation transfers the state of a qubit from one place to another using a shared entangled pair and two classical bits of communication. The sender measures their qubits and sends the classical results; the receiver applies correction gates based on those results to recover the original state.

```

place Alice {
    println("Alice is preparing to teleport a qubit to Bob...");
    // Prepare Bell pair
    state q1, q2;
    H q1;
    CNOT q1 -> q2;
    // Send one qubit to Bob
    send q2 to Bob as "qubit";
    println("Bell pair prepared. One qubit sent to Bob.");

    // Get one bit of input from user
    state to_send;
    num flip;
    print("Enter a bit to teleport (0 or 1): ");
    input(flip, 0..1);
    println("Input '%d' received. Preparing qubit to teleport..." % flip);

    // 0 = no change, 1 = apply X gate
    if (flip == 1) X to_send;

    // Prepare qubit to teleport
    CNOT to_send -> q1;
    H to_send;

    // Measure both
    obs measurements[2] = measure [to_send, q1];

    // Send measurement results to Bob
    println("Measurement complete: m0=%d m1=%d" % [measurements[0], measurements[1]]);
    send measurements to Bob as "measurements";
}

```



```

println("Measurement results sent to Bob.");
}

place Bob {
    // Receive qubit from Alice (half of Bell pair)
    println("Waiting for qubit from Alice...");
    state q2 = receive state from Alice named "qubit";
    println("Received qubit from Alice.");

    println("Waiting for measurement results from Alice...");
    obs measurements[2] = receive obs from Alice named "measurements";
    println("Received measurement results from Alice.");

    // Apply corrections based on Alice's measurements
    if(measurements[0] == 1) Z q2;
    if(measurements[1] == 1) X q2;

    // q2 should now be in the same state as Alice's original to_send qubit
    println("Teleportation complete.");
    obs orig_state = measure q2;

    println("Original state (0 or 1): %d" % orig_state);
}

```

3.5 Example: Superdense Coding

Superdense coding allows two classical bits of information to be transmitted using only one qubit, provided the sender and receiver share an entangled pair beforehand. The sender encodes the two bits by applying gates to their half of the pair and sends the qubit; the receiver decodes both bits by measuring the pair jointly.

```

place Alice{
    // Prepare entangled pair and send one qubit to Bob
    state qA, qB;
    H qA;
    CNOT qA -> qB;
    send qB to Bob as "qubitB";

    // Get two bits of input from user
    obs bits[2];
    print("Enter number 0-3 (two bits): ");
    input(bits, 0..3);
    println("Input '%d' received. Preparing qubit to send..." % bits);
    println("Bits to send: %d %d" % [bits[0], bits[1]]);
}

```

```

// Encode the two bits into the qubit
if (bits[0]) Z qA; // Flip phase for bit 0
if (bits[1]) X qA; // Flip bit for bit 1

// Send the encoded qubit to Bob
send qA to Bob as "qubitA";
println("Encoded qubit sent to Bob.");
}

place Bob {
    println("Bob is waiting for qubits from Alice...");
    state qB = receive state from Alice named "qubitB";
    println("Received first qubit from Alice.");
    state qA = receive state from Alice named "qubitA";
    println("Received second qubit from Alice.");

    // Decode the received qubits
    CNOT qA -> qB;
    H qA;
    obs result[?] = measure [qA, qB];
    println("Measurement complete. Decoded bits: %d %d" % [result[0], result[1]]);
    println("Decoded number: %d" % result);
}

```

4 Advanced Control Flow and Scope Management

4.1 The Parent Operator ^ and Scope Hierarchy

4.1.1 Scope in QLang

In QLang, every block of code creates a new scope level, forming a linked chain of scopes. Reading a variable walks this chain upward automatically. Writing to a variable in an outer scope requires the parent operator ^.

4.1.2 How ^ Works

The parent operator ^ explicitly selects a scope one level above the current one. Multiple carets chain upward, one level per caret:

- ^X accesses X in the parent scope
- ^^X accesses X in the grandparent scope
- ^^^X accesses X three levels up

Once the target scope is reached, the variable is looked up only in that specific scope; the automatic upward walk does not apply at that point. The operator supports all standard operations: assignment, compound assignment, increment and decrement, as well as indexed access on arrays.

The following example demonstrates reading and writing through the scope hierarchy:

```
num x = 1;
if (1) {
    num x = 2;
    if (1) {
        println("%d" % ^x);    // 2 - reads parent x
        ^x = 7;
        println("%d" % ^x);    // 7 - assigns to parent x
        (^x)++;
        println("%d" % ^x);    // 8
        ^x += 3;
        println("%d" % ^x);    // 11
        println("%d" % ^^x);   // 1 - reads grandparent x
    }
}
```

Array indexing works the same way. The parent operator applies to the variable, and the index is evaluated afterward:

```
num arr[2] = [10, 20];
if (1) {
    if (1) {
        println("%d" % ^^arr[0]); // 10
        println("%d" % ^^arr[1]); // 20
    }
}
```

```
}  
}
```

4.2 Advanced Array Structures and Indexing

QLang supports three indexing modes for arrays: simple indexing, range indexing, and list indexing. All three resolve negative indices automatically against the container length.

4.2.1 Simple Indexing and Negative Indices

A single integer index selects one element. Negative values count from the end of the array: an index i where $i < 0$ is resolved as $i + \text{length}$ before access.

```
num arr[5] = [10, 20, 30, 40, 50];  
println(arr[0]);    // 10  
println(arr[-1]);   // 50  
println(arr[-2]);   // 40
```

4.2.2 Range Indexing

A range index **start..end** selects a contiguous subarray. The range is left-inclusive and right-exclusive: element at **start** is included, element at **end** is not. Negative values are resolved the same way as in simple indexing.

```
num arr[5] = [10, 20, 30, 40, 50];  
println(arr[1..3]); // [20, 30]  
println(arr[0..-1]); // [10, 20, 30, 40]
```

4.2.3 List Indexing

A list index **[[a, b, c]]** selects elements at the given positions in arbitrary order. The double bracket syntax distinguishes list indexing from simple indexing. This is particularly useful for selecting non-contiguous elements or reordering array contents.

```
num arr[5] = [10, 20, 30, 40, 50];  
println(arr[[0, 2, 4]]); // [10, 30, 50]  
println(arr[[4, 0, 2]]); // [50, 10, 30]
```

An existing array can also be passed as the index list directly:

```
num arr[5] = [10, 20, 30, 40, 50];  
num idx[3] = [0, 2, 4];  
println(arr[idx]); // [10, 30, 50]
```

4.3 ...